

Computer Networks

“Final Project Report:

ChatFlow: *Encrypted Bluetooth Mesh Chat for Nearby Devices*

Submitted By

Abdul Rahman Azam

23K-0061

Hafsa Rashid

23K-0064

FAST School of Computing

National University of Computer & Emerging Sciences

1. Motivation

In many situations, internet is simply not available. For example in disaster areas, remote locations, or crowded events where the network is overloaded, normal messaging apps stop working completely because they need a server and internet connection. ChatFlow solves this problem by using Bluetooth Low Energy (BLE), which is already present in every modern smartphone, so nearby devices can chat directly without any internet or Wi-Fi. All messages are encrypted with AES-128 so even if someone is listening on the channel, they cannot read the messages.

2. Overview

ChatFlow is an Android app that allows nearby devices to exchange encrypted messages over Bluetooth Low Energy without needing internet. Each device acts as both a sender and a receiver at the same time. Messages can also pass through middle devices to reach someone who is not in direct Bluetooth range, making it a simple mesh network. Every message has a unique ID and a hop limit so that it does not keep circulating in the network forever.

2.1 Significance of the Project

This project covers many important Computer Networks topics in one working system, including wireless communication, ad-hoc network topology, routing, message fragmentation, and encryption. It is useful in real life situations like disaster response, military field operations, or any place where internet is not available. The whole project was built using free Android tools with no paid services or external servers needed.

2.2 Description of the Project

ChatFlow is an Android application for sending and receiving encrypted messages over Bluetooth Low Energy without internet. Each phone scans for nearby devices while also broadcasting its own presence so other phones can find it. If a message needs to travel more than one hop, middle devices forward it to their other connected peers. Large messages are broken into smaller numbered pieces and put back together at the receiving end. All messages are encrypted using AES-128-CBC before sending and decrypted only after full reassembly at the destination. Every message carries a unique UUID and a TTL value. If a device receives a message it has already seen, it simply ignores it to stop forwarding loops.

2.3 Background of the Project

Bluetooth Low Energy is a wireless technology supported on Android phones since Android 5.0 (API level 21). BLE uses GATT Services and Characteristics to send and receive data between devices. ChatFlow is based on the idea of Bluetooth mesh networking where phones forward messages for each other without any central router or server. AES (Advanced Encryption Standard) is a well known encryption method approved by NIST and is used worldwide to protect data. Android provides built-in AES support through the javax.crypto package so no extra library is needed. The Bluetooth Core Specification defines important link layer limits like MTU size and

advertising payload limits which directly affect how ChatFlow handles message fragmentation and device discovery .

2.4 Project Category

ChatFlow is a Product-Based project. It produces a fully working Android APK that can be installed and used on real phones to send encrypted messages without internet. The project is made for academic purposes but can be practically used in any situation where internet is not available.

3. Features / Scope / Modules

ChatFlow is divided into three modules. Features are listed from most important to least important.

- **BLE Mesh Discovery and Multi-Device Connection.** Each phone advertises itself as a GATT server and scans for other nearby GATT servers at the same time. It can maintain connections with multiple peers together and shows connection status in real time on the screen.
- **Flood-Based Mesh Relay with Loop Prevention.** When a device receives a message that is not meant for it, it forwards that message to all its other connected peers after reducing the TTL by one. A list of already seen message UUIDs is kept so that the same message is never forwarded twice, which stops infinite loops in the network.
- **AES-128-CBC End-to-End Encryption.** All messages are encrypted before being sent over Bluetooth and decrypted only after full reassembly at the destination. The standard javax.crypto library is used with no third party dependency. A new random IV is generated for every message so the same text always produces a different ciphertext.
- **Message Fragmentation and Reassembly.** Messages bigger than the BLE MTU limit of 512 bytes are first encrypted and then split into smaller numbered chunks. The receiving device collects all chunks and reassembles them in correct order before decrypting, even if some chunks arrive out of order.
- **Application-Layer Message Protocol.** Every message frame contains the sender ID, recipient ID, message UUID, timestamp, TTL value, chunk index, total chunks, IV, and the encrypted payload. This information is enough for routing, loop prevention, and decryption without needing any central server.
- **Real-Time Chat UI and Local Chat History.** The app shows chat bubbles with sender name and time. All conversations are saved locally using a Room SQLite database so chat history is available even after restarting the app. A foreground service keeps Bluetooth connections alive when the app is running in the background.
- **Automatic Reconnection.** If a peer disconnects unexpectedly, the app automatically tries to reconnect using exponential backoff so the user does not need to reconnect manually.

4. Project Planning

The project was executed over five weeks. The table below details the planned and actual schedule for each task, the responsible team member, and completion status.

Task / Activity	Planned	Actual	Owner	Done
Requirement Analysis, Architecture & UI Mockups	Week 1	Week 1	Both	Yes
BLE Device Manager: Scanning, GATT, Multi-Device	Week 2	Week 2	Abdul Rahman	Yes
Chat Manager: AES Encryption, Framing, Mesh Relay	Week 3	Week 3	Hafsa Rashid	Yes
UI, Chat History, Room DB & Notifications	Week 4	Week 4	Abdul Rahman	Yes
Integration, Multi-Device Testing & Debugging	Week 5	Week 5	Both	Yes
Documentation & Final Report	Week 5	Week 5	Both	Yes

Table 1. Project schedule with planned vs. actual timeline, ownership, and completion status.

5. Project Feasibility

5.1 Technical Feasibility

All the required APIs including BLE GATT dual role, AES encryption via javax.crypto, and the Room database library are already part of the standard Android SDK from API level 21. No special hardware is needed beyond a normal Android phone. The dual role BLE feature was tested and confirmed working on the devices used. One possible risk is that some phone manufacturers have slightly different BLE behaviour, but this is handled by the automatic reconnection with exponential backoff.

5.2 Economic Feasibility

The project has zero cost. Android Studio, Kotlin, and Gradle are all free. There is no need for any cloud service, paid SDK, or server. The only hardware needed is Android phones which the team already had.

5.3 Schedule Feasibility

Five weeks was enough time for this project. Because the project was divided into three clear modules, both team members could work on separate parts at the same time without depending on each other. All tasks were finished on time as shown in Table 1.

6. Hardware and Software Requirements

6.1 Hardware

- **Android Devices:** At least two Android phones running Android 5.0 or above with BLE support for simultaneous peripheral and central roles. A third phone is needed to test the multi-hop mesh relay feature.
- **Development Machine:** Any laptop or desktop that can run Android Studio.

6.2 Software

- **Android Studio (Hedgehog or later):** Primary IDE and build environment.
- **Kotlin 1.9:** Implementation language.
- **Android SDK (API 21+):** BLE GATT APIs and standard cryptography libraries.
- **Room Persistence Library:** SQLite database abstraction for chat history storage.
- **javax.crypto (standard Android SDK):** AES-128-CBC encryption and decryption; no additional library required.

7. Diagrammatic Representation of the Overall System

The following screenshots are from the working application demonstrating the complete user flow from startup through to active encrypted chat sessions.

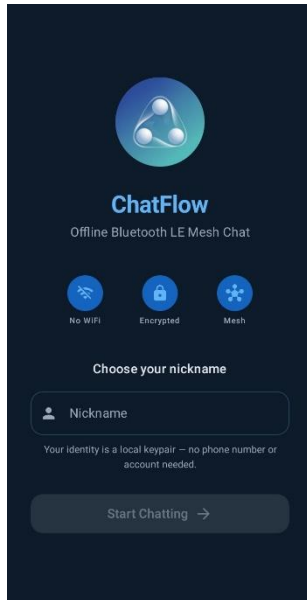


Figure 1: App startup with nickname entry.

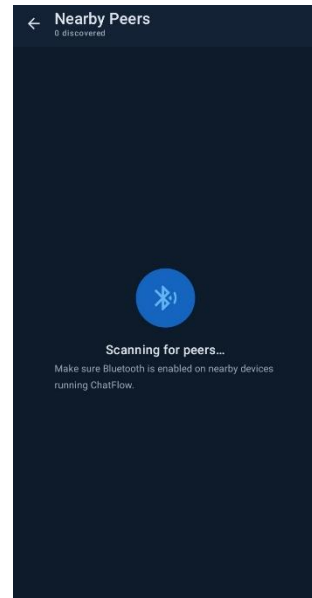
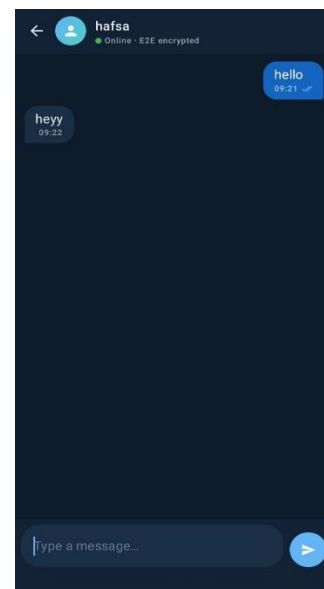


Figure 2: BLE scanning for nearby peers.



Figures 3-4: Peer discovered and active E2E encrypted chat

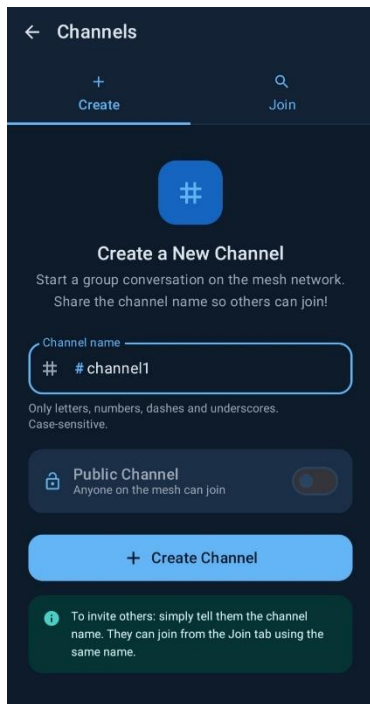


Figure 5: Creating a new named channel

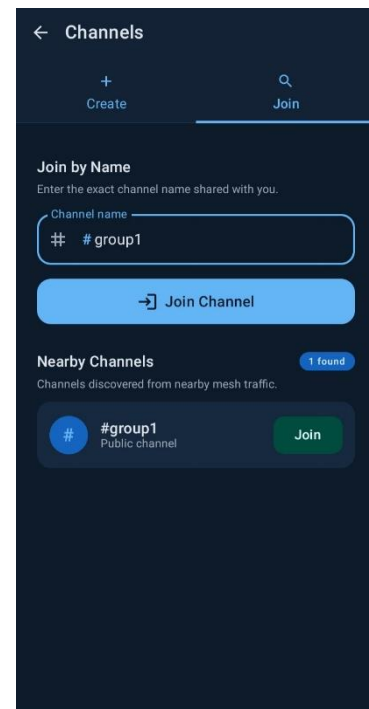


Figure 7: Searching and joining for channel

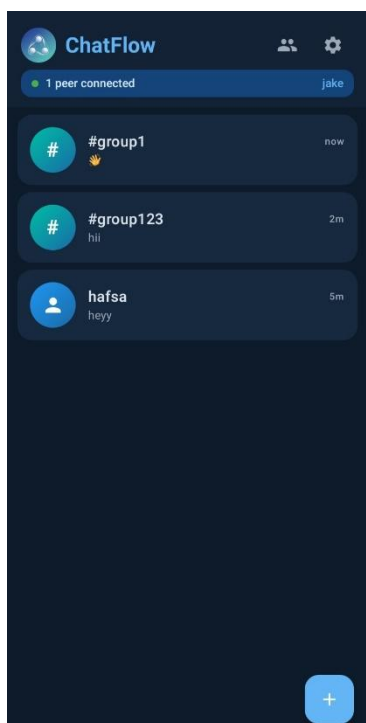


Figure 8: Main sessions view showing all active chats and connected peers.

8. References

1. Android Bluetooth Guide – Official Android documentation
<https://developer.android.com/guide/topics/connectivity/bluetooth>
2. Bluetooth Low Energy (BLE) Overview – Official Android documentation
<https://developer.android.com/guide/topics/connectivity/bluetooth-le>
3. AES Encryption in Java/Kotlin – Oracle official cryptography guide
<https://docs.oracle.com/javase/8/docs/technotes/guides/security/crypto/CryptoSpec.html>